RAG 교안 05 · 7부~9부

질문으로 근거 찾기와 검색 결과 확인

앞 교안에서 저장한 청크를 검색하고, 원문 근거와 검색 결과를 확인합니다 (검색·문항·검사 노트북)

검색 노트북 7부 · 질문 한 건의 근거 검색 · 앞 교안(저장 노트북)의 DB와 설정 사용
문항 노트북 8부 · 질문과 정답 CSV 준비 · 원문을 확인한 20문항
검사 노트북 9부 · 검색 결과 검사 · 근거 순위와 발견 비율 확인

수업 순서 준비(2~5쪽) → 근거 검색(6~24쪽) → 평가·검사(25~42쪽) → 활동·마무리(43~49쪽)

앞 교안에서 저장한 자료를 검색에 이어 씁니다

앞 교안(04.RAG_저장)의 저장 노트북는 글·숫자·출처와 설정을 남겼습니다. 이 교안은 앞 교안에서 예고한 검색 노트북에 평가용 문항·검사 노트북을 더해, 그 자료로 질문에 맞는 근거를 찾고 검사합니다.

연결

앞 교안에서 남긴 것	이 교안에서 쓰는 곳	역할
chroma_db / skhynix_basic	검색 노트북 코드 3 · 검사 노트북 코드 3	저장된 글·숫자·출처를 다시 읽습니다.
artifacts/index.json	검색 노트북 코드 2 · 검사 노트북 코드 2	모델·컬렉션·지시문·청크 수를 가져옵니다.
metadata의 period	검색 노트북 코드 6 · 검사 노트북 코드 9	보고기간이 같은 청크로 검색 범위를 좁힙니다.

학습 순서

검색 노트북은 한 질문의 후보와 원문을 읽습니다.
문항 노트북은 20문항의 질문·정답·필수 근거를 CSV로 저장합니다.
검사 노트북은 후보에서 근거를 찾은 순위와 전체 비율을 확인합니다.

연결

저장 노트북에서 만든 변수는 새 노트북으로 자동 전달되지 않습니다. DB와 설정 파일을 다시 읽습니다.

다음 → 누가 무엇을 하는지 구분합니다

누가 무엇을 하는지 구분합니다

앞 교안과 마찬가지로 프로그램이 요청하고, 임베딩 모델이 숫자를 만들며, ChromaDB가 저장된 자료를 돌려줍니다.

이름	맡은 일
Python 프로그램	설정을 읽고, 임베딩 요청을 보내고, 검색 응답을 표로 만들고, 근거를 비교합니다.
Ollama	이 PC에서 임베딩 모델을 실행하고 Python 프로그램의 요청을 받습니다.
임베딩 모델(qwen3-embedding:0.6b)	입력 글을 서로 비교할 숫자 목록으로 바꿉니다.
ChromaDB	저장된 청크에서 기간 조건에 맞는 검색 후보와 원문·출처를 반환합니다.
Jupyter 커널	선택한 Python 환경(stock_agent_env01)에서 노트북의 코드 셀을 실행합니다.
생성 모델(후속 구성)	후속 RAG 구성에서 근거를 참고해 답변 문장을 작성할 수 있습니다. 현재 노트북에는 생성 호출이 없습니다.

목표 질문·기간을 지정하고, 원문의 출처·열 제목·단위를 확인하며, 검색 결과 지표의 뜻을 설명합니다.

다음 → 실행할 커널과 작업 폴더를 확인합니다

실행할 커널과 작업 폴더를 확인합니다

앞 교안에서 사용한 Python 환경과 Ollama 앱을 이어 씁니다. 아래 화면은 환경을 확인한 진단 결과입니다.

실행 결과

```
[1]: import sys, platform
     print(sys.executable)
     print(sys.version)
     print(platform.platform())

C:\ProgramData\anaconda3\envs\stock_agent_env01\python.exe
3.11.0 | packaged by Anaconda, Inc. | (main, Mar  1 2023, 18:18:21) [MSC v.1916 64 bit (AMD64)]
Windows-10-10.0.26200-SP0
```

진단 화면 1 · 환경 확인 · 실행 결과

확인할 점

커널

Python (stock_agent_env01)을 선택합니다.

작업 폴더

Path.cwd()가 자신의 rag_exam을 가리키는지 봅니다.

모델

qwen3-embedding:0.6b가 준비되어 있는지 확인합니다.

구분

진단 화면은 교안의 보충 자료입니다. 검색 노트북에서 실행할 코드와 구분해서 봅니다.

다음 → 저장한 청크 수와 검색 설정을 확인합니다

저장한 청크 수와 검색 설정을 확인합니다

앞 교안의 저장 결과를 다시 확인합니다. skhynix_basic에 800개가 있고, 이 중 2025Q3 청크는 203개입니다.

실행 결과

```
        settings = {'model': 'qwen3-embedding:0.6b', 'collection': 'skhynix_basic', 'query_prefix': 'Instruc
        t: Given a question, retrieve passages from Korean financial disclosures that answer the question.\nQ
        uery: ', 'chunk_count': 800}
        collection.count() = 800
        2025FY 202
        2025Q3 203
        2026Q1 188
        2026H1 199
        20260722 8
        collection.metadata = None
        embedding length of 2025Q3_0196 = 1024
        {'source': '000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md', 'section': 'III. 재무에 관한 사항 > 2-
        2. 연결 포괄손익계산서', 'period': '2025Q3'}
```

진단 화면 3 · DB 확인 · 실행 결과

확인할 점

설정 파일

index.json에는 model, collection, query_prefix, chunk_count가 있습니다.

DB

글·숫자·출처는 chroma_db에 있습니다.

비교 기준

cosine 거리, 숫자 목록 길이 1,024개입니다.

연결

800은 전체 청크 수, 203은 기간이 같은 청크 수, 1,024는 임베딩 숫자의 개수입니다.

다음 → 프로그램이 파일을 찾을 폴더를 정합니다

프로그램이 파일을 찾을 폴더를 정합니다

프로그램은 현재 작업 폴더 안의 data와 artifacts를 가리키는 이름을 만듭니다. 앞 교안 저장 노트북 코드 1과 같은 준비입니다.

실행 결과

```
[1]: # 폴더와 파일의 위치를 다루는 Path 도구를 가져옵니다. import는 준비된 도구를 불러온다는 뜻입니다.
from pathlib import Path

# 현재 작업 폴더 안의 data를 가리킵니다. =는 오른쪽 값을 왼쪽 이름에 담으라는 뜻입니다.
data_folder = Path("data")
# 프로그램이 만든 청크·설정·테스트 결과를 둘 위치입니다. 이 줄만으로 폴더가 생기지는 않습니다.
output_folder = Path("artifacts")
# 현재 작업 폴더의 전체 경로를 반환합니다. 마지막 줄의 값은 Jupyter가 화면에 보여 줍니다.
Path.cwd()

[1]: WindowsPath('D:/교육자료/한국폴리텍대학교_하이테크_2026_2학기/work/rag_search_rewrite_20260921/run/rag_exam')
```

검색 노트북 코드 1 · 실행 결과 (셀 번호 [n]은 실행 순서이며 교안의 코드 번호와 다릅니다)

확인할 점

상대경로

Path("data")와 Path("artifacts")는 현재 위치를 기준으로 합니다.

출력

Path.cwd()는 현재 폴더의 전체 위치입니다.

확인

이 셀은 폴더를 만들거나 파일 안의 글을 읽지 않습니다.

주의 화면의 경로는 기존 실행 복사본입니다. 자신의 환경에서는 rag_exam 폴더인지 확인합니다.

다음 → index.json을 읽어 설정을 다시 가져옵니다

index.json을 읽어 설정을 다시 가져옵니다

프로그램은 artifacts/index.json을 열고, json.load로 읽은 네 설정값을 settings에 담습니다.

실행 결과

```
[2]: # 설정 파일의 JSON 내용을 읽을 도구입니다.
import json

# 02에서 저장한 설정 파일을 읽기 모드로 엽니다.
with (output_folder / "index.json").open(encoding="utf-8") as file:
    # 파일의 값을 이름으로 꺼낼 수 있는 딕셔너리로 받습니다.
    settings = json.load(file)
```

검색 노트북 코드 2 · 실행 결과

확인할 점

입력

앞 교안 저장 노트북 코드 11이 저장한 index.json입니다.

결과

모델 이름, 컬렉션 이름, 질문용 지시문, 기대 청크 수를 읽습니다.

파일이 없으면

저장 노트북 코드 11 실행 여부와 현재 작업 폴더를 확인합니다.

연결

파일을 읽어야 settings를 사용할 수 있습니다. 저장 노트북의 메모리 변수는 공유되지 않습니다.

다음 → 저장해 둔 skhynix_basic 컬렉션을 엽니다

저장해 둔 skhynix_basic 컬렉션을 엽니다

프로그램은 chroma_db의 전체 경로로 DB에 연결하고, 설정 파일에 적힌 이름의 기존 컬렉션을 엽니다.

실행 결과

```
[3]: # 디스크에 원문과 임베딩을 저장하고 검색하는 ChromaDB 도구입니다.
import chromadb

# chroma_db라는 상대경로의 실제 전체 위치를 실행 중 계산합니다.
db_folder = Path("chroma_db").resolve()
# 그 위치의 DB를 사용하기 위한 연결 객체를 만듭니다. Persistent는 디스크에 남긴다는 뜻입니다.
client = chromadb.PersistentClient(path=str(db_folder))
# 설정 파일에 적힌 기존 컬렉션을 엽니다. 자동 임베딩은 사용하지 않습니다.
collection = client.get_collection(settings["collection"], embedding_function=None)
```

검색 노트북 코드 3 · 실행 결과

확인할 점

PersistentClient
디스크의 DB에 연결합니다.

get_collection
기존 컬렉션을 엽니다. 없는 이름이면 오류가 납니다.

embedding_function=None
질문 임베딩은 프로그램이 Ollama에 직접 요청합니다.

연결

앞 교안은 컬렉션을 준비해 저장했고, 이 교안은 기존 800개를 읽습니다. 문서 임베딩을 다시 계산하지 않습니다.

다음 → 질문과 보고기간을 각각 지정합니다

질문과 보고기간을 각각 지정합니다

question에 찾을 내용을, period에 조회할 보고기간을 따로 넣습니다. 프로그램이 질문에서 기간을 자동으로 추출하지 않습니다.

실행 결과

```
[4]: # 찾고 싶은 내용을 질문 문장으로 적습니다. 정답 금액을 검색어에 넣지 않습니다.
question = "2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?"
# 사용자가 선택하는 기간 조건입니다. 질문을 자동 분석해 만든 값이 아닙니다.
period = "2025Q3"
```

검색 노트북 코드 4 · 실행 결과

question에는 질문만 넣고 정답 금액은 넣지 않습니다. period는 아래 이름표 중 하나입니다.

보고기간 이름표(period)	2025FY	2025Q3	2026Q1	2026H1	20260722
뜻	2025년 연간	2025년 3분기	2026년 1분기	2026년 반기	시설투자 공시

주의 질문이나 기간을 바꾸면 검색 노트북 코드 5(임베딩)부터 코드 9(원문 표시)까지 순서대로 다시 실행합니다.

다음 → 저장한 질문용 지시문을 다시 확인합니다

저장한 질문용 지시문을 다시 확인합니다

앞 교안 저장 노트북 코드 11에서 정한 query_prefix입니다. 프로그램은 이 지시문 뒤에 질문을 붙여 임베딩 모델에 보냅니다.

원본 코드

```
[11]: # 검색 목적을 알려 주는 공통 글입니다. Query: 뒤에는 실제 질문이 붙습니다.
query_prefix = "Instruct: Given a question, retrieve passages from Korean financial disclosures that answer the question.\nQuery: "
```

저장 노트북 코드 11 · 1~2행 · 원본 코드

지시문은 한국어 재무 공시에서 질문에 답하는 글을 찾도록 안내합니다. 실제 문자열에는 줄바꿈과 Query: 뒤 공백이 있습니다.

모델에 보내는 글의 구조

Instruct: Given a question, retrieve passages from Korean financial disclosures that answer the question.
Query: 2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?

연결

검색 노트북은 index.json에서 이 값을 읽습니다. 확인을 위해 저장 노트북 전체를 다시 실행할 필요는 없습니다.

다음 → 검색에 이어 쓰는 네 설정값을 확인합니다

검색에 이어 쓰는 네 설정값을 확인합니다

앞 교안에서 settings를 index.json에 저장한 부분입니다. 검색 노트북과 검사 노트북은 이 파일에서 같은 모델과 지시문을 읽습니다.

원본 코드

```
# 다른 노트북이 같은 설정을 읽도록 필요한 값만 한 묶음으로 만듭니다.
settings = {
    # 사용한 모델과 저장한 보관함의 이름입니다.
    "model": model, "collection": collection_name,
    # 질문용 공통 지시문과 기대하는 저장 개수입니다.
    "query_prefix": query_prefix, "chunk_count": len(chunks)
}
# 설정 결과 파일을 엽니다. 다음 노트북은 이 파일을 읽습니다.
with (output_folder / "index.json").open("w", encoding="utf-8") as file:
    # 한글을 유지하고 들여쓰기 두 칸으로 사람이 읽기 편하게 저장합니다.
    json.dump(settings, file, ensure_ascii=False, indent=2)
```

저장 노트북 코드 11 · 3~13행 · 원본 코드

확인할 점

model
qwen3-embedding:0.6b
collection
skhynix_basic
query_prefix
질문 앞에 붙일 검색 지시문
chunk_count
800

구분 index.json은 설정 파일입니다. 청크의 글과 임베딩 숫자는 chroma_db의 DB에 있습니다.

다음 → 질문을 보내고 숫자 목록을 받습니다

질문을 보내고 숫자 목록을 받습니다

프로그램은 검색 지시문과 질문을 합쳐 Ollama에 보냅니다. 임베딩 모델은 그 글에 대응하는 숫자 목록을 만듭니다.

실행 결과

```
[5]: # 문서 임베딩 때와 같은 Ollama 요청 도구입니다.
import ollama

# 질문 임베딩 요청의 결과를 받습니다.
response = ollama.embed(
    # 02와 같은 모델을 쓰고, 지시문과 질문만 합쳐 보냅니다.
    model=settings["model"], input=settings["query_prefix"] + question,
    # 입력이 너무 길면 조용히 잘라내지 않도록 합니다.
    truncate=False
)
# 질문 하나에 대응하는 첫 숫자 목록을 꺼냅니다.
query_vector = response.embeddings[0]
```

검색 노트북 코드 5 · 실행 결과

확인할 점

같은 모델
저장할 때 쓴 settings["model"]을 사용합니다.

query_vector
첫 입력의 숫자 목록을 담습니다.

truncate=False
입력이 너무 길면 조용히 자르지 않고 오류로 알립니다.

연결

앞 교안은 청크의 글을, 이 교안은 지시문과 질문을 임베딩합니다. 1,024는 기존 진단 결과의 길이입니다.

다음 → period가 같은 청크로 검색 범위를 좁힙니다

period가 같은 청크로 검색 범위를 좁힙니다

ChromaDB는 where 조건으로 period가 2025Q3인 청크 203개에서 후보를 찾습니다. 나머지 597개는 이번 검색에서 제외합니다.

실행 결과

```
[6]: # 검색 응답 전체를 result라는 이름으로 받습니다.
result = collection.query(
    # 질문 한 개의 숫자 목록을 다시 목록으로 감싸 보냅니다. where는 기간 필터입니다.
    query_embeddings=[query_vector], where={"period": period},
    # 최대 다섯 개의 원문·이름표·거리를 돌려달라고 요청합니다.
    n_results=5, include=["documents", "metadatas", "distances"]
)
```

검색 노트북 코드 6 · 실행 결과

203개는 기간이 같은 검색 대상의 수입니다. 이 값은 정답 개수가 아닙니다.

period 이름표	2025FY	2025Q3	2026Q1	2026H1	20260722	전체
청크 수(현재 자료)	202	203	188	199	8	800

연결

앞 교안에서 metadata에 저장한 period가 여기서 조건으로 쓰입니다. 잘못 지정하면 필요한 근거를 제외할 수 있습니다.

다음 → 질문의 숫자와 검색 조건을 함께 전달합니다

질문의 숫자와 검색 조건을 함께 전달합니다

collection.query 한 번의 호출이 보고기간 조건과 후보 개수 요청을 함께 처리합니다.

인자	값	뜻
query_embeddings	[query_vector]	질문 한 건의 숫자 목록을 목록으로 감싸 전달합니다. 바깥 목록은 여러 질문을 묶는 자리입니다.
where	{"period": period}	사람이 정한 보고기간 조건입니다.
n_results	5	최대 5개 후보를 요청합니다. 정답 5개나 서로 다른 문서 5개라는 뜻이 아닙니다.
include	["documents", "metadatas", "distances"]	원문, 이름표(출처·기간·절), 거리를 함께 받습니다. ID는 항상 돌아옵니다.

[6]:
```
# 검색 응답 전체를 result라는 이름으로 받습니다.
result = collection.query(
    # 질문 한 개의 숫자 목록을 다시 목록으로 감싸 보냅니다. where는 기간 필터입니다.
    query_embeddings=[query_vector], where={"period": period},
    # 최대 다섯 개의 원문·이름표·거리를 돌려달라고 요청합니다.
    n_results=5, include=["documents", "metadatas", "distances"]
)
```

ChromaDB는 조건에 맞는 청크 중 질문의 숫자 목록과 가까운 후보를 최대 5개 돌려줍니다.

검색 노트북 코드 6 · 실행 결과

구분 5는 요청할 후보 청크 수입니다. 정답 5개 또는 서로 다른 원문 파일 5개라는 뜻은 아닙니다.

다음 → 응답의 같은 번호끼리 한 후보로 묶습니다

응답의 같은 번호끼리 한 후보로 묶습니다

검색 응답 result에는 ID, 원문, 이름표, 거리가 각각 목록으로 들어 있습니다. 프로그램은 같은 위치의 값을 함께 꺼냅니다.

실행 결과

```
[7]: # 화면에 보여 줄 결과표의 행들을 모을 목록입니다.
rows = []
# 첫 질문의 결과 수만큼 0, 1, 2, ... 순서로 반복합니다.
for number in range(len(result["ids"][0])):
    # 이번 결과 번호에 해당하는 출처와 제목을 꺼냅니다.
    metadata = result["metadatas"][0][number]
    # 같은 번호의 정보들을 하나의 결과 행으로 모읍니다.
    rows.append({
```

검색 노트북 코드 7 · 1~8행 · 실행 결과

확인할 점

같은 번호
같은 후보의 ID·원문·이름표·거리입니다.

[0]
첫 질문의 결과 목록입니다.

number
후보가 5개이면 0~4입니다.

metadata
source, period, section입니다.

연결

저장할 때 같은 청크의 정보를 같은 순서로 모았듯, 검색 응답도 같은 번호의 값을 연결합니다.

다음 → 후보 하나의 정보를 결과표 한 행에 담습니다

후보 하나의 정보를 결과표 한 행에 담습니다

프로그램은 순위, 청크 ID, 거리, 출처, 절, 근거를 한 행으로 만듭니다. 같은 셀의 뒷부분입니다.

실행 결과

```
        # 사람에게 보이는 순위는 1부터, 청크 ID는 저장된 이름 그대로입니다.
        "순위": number + 1, "청크ID": result["ids"][0][number],
        # 작을수록 질문 숫자 목록과 가까운 거리입니다. 출처는 원문 파일명입니다.
        "거리": result["distances"][0][number], "출처": metadata["source"],
        # 제목과 실제 원문 근거도 함께 넣습니다.
        "절": metadata["section"], "근거": result["documents"][0][number]
    })
```

검색 노트북 코드 7 · 9~15행 · 실행 결과

확인할 점

순위
number + 1로 1부터 셉니다.

거리
작을수록 질문의 숫자 목록과 가깝습니다.

출처·절
원문 파일명과 절 제목입니다.

근거
DB에 저장된 원문 조각입니다.

구분

표의 행 번호 0과 검색 순위 1은 다릅니다. 거리의 크기는 정답 확률을 뜻하지 않습니다.

다음 → 검색한 다섯 후보를 표로 확인합니다

검색한 다섯 후보를 표로 확인합니다

프로그램은 hits에서 긴 근거 열을 잠시 빼고 순위·청크 ID·거리·출처·절을 보여 줍니다.

실행 결과

[8]:
```
# 목록을 표로 다루는 pandas 도구를 pd라는 짧은 이름으로 불러옵니다.
import pandas as pd

# 딕셔너리 하나를 한 행으로, 이름들을 열 제목으로 만듭니다.
hits = pd.DataFrame(rows)
# 긴 원문을 잠시 빼고 순위·ID·거리·제목·출처를 표시합니다.
hits[["순위", "청크ID", "거리", "절", "출처"]]
```

[8]:

	순위	청크ID	거리	절	
0	1	2025Q3_0196	0.177438	III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서	000660_SK하이닉스_2025Q3_원문
1	2	2025Q3_0137	0.185091	II. 사업의 내용 > 7. 기타 참고사항	000660_SK하이닉스_2025Q3_원문
2	3	2025Q3_0142	0.197844	II. 사업의 내용 > 7. 기타 참고사항	000660_SK하이닉스_2025Q3_원문
3	4	2025Q3_0193	0.207058	III. 재무에 관한 사항 > 1. 요약재무정보	000660_SK하이닉스_2025Q3_원문
4	5	2025Q3_0014	0.221572	II. 사업의 내용 > 2. 주요 제품 및 서비스	000660_SK하이닉스_2025Q3_원문

검색 노트북 코드 8 · 실행 결과

확인할 점

2026-09-21 실행 순위
1 2025Q3_0196
2 2025Q3_0137
3 2025Q3_0142
4 2025Q3_0193
5 2025Q3_0014
모두 같은 2025Q3 보고서의 청크입니다.

확인 같은 문서에서 여러 청크를 찾을 수 있습니다. 표에서 빠진 근거 열도 hits에는 남아 있습니다.

다음 → 거리와 검색 순위를 함께 읽습니다

거리와 검색 순위를 함께 읽습니다

작은 거리는 질문의 숫자 목록과 더 가까운 후보라는 뜻입니다. 아래 표는 기존 두 실행의 값을 구분해서 보여 줍니다.

실행 결과

[8]:

	순위	청크ID	거리	절	
0	1	2025Q3_0196	0.177438	III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서	000660_SK하이닉스_2025Q3_원문
1	2	2025Q3_0137	0.185091	II. 사업의 내용 > 7. 기타 참고사항	000660_SK하이닉스_2025Q3_원문
2	3	2025Q3_0142	0.197844	II. 사업의 내용 > 7. 기타 참고사항	000660_SK하이닉스_2025Q3_원문
3	4	2025Q3_0193	0.207058	III. 재무에 관한 사항 > 1. 요약재무정보	000660_SK하이닉스_2025Q3_원문
4	5	2025Q3_0014	0.221572	II. 사업의 내용 > 2. 주요 제품 및 서비스	000660_SK하이닉스_2025Q3_원문

읽을 때 확인

행 번호 0~4와 검색 순위 1~5를 구분합니다. 0.177이라는 거리를 정확도 82%로 읽지 않습니다.

검색 노트북 코드 8 · 실행 결과

순위	청크 ID	절	2026-09-21 거리	기존 저장 기록 거리
1	2025Q3_0196	III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서	0.177438	0.178247
2	2025Q3_0137	II. 사업의 내용 > 7. 기타 참고사항	0.185091	0.185084
3	2025Q3_0142	II. 사업의 내용 > 7. 기타 참고사항	0.197844	0.198972
4	2025Q3_0193	III. 재무에 관한 사항 > 1. 요약재무정보	0.207058	0.210467
5	2025Q3_0014	II. 사업의 내용 > 2. 주요 제품 및 서비스	0.221572	0.218832

확인

두 실행의 청크 ID와 순서는 같고 거리의 소수값만 다릅니다. 정답은 원문에서 확인합니다.

다음 → 1위 청크의 원문을 화면에 표시합니다

1위 청크의 원문을 화면에 표시합니다

프로그램은 hits의 첫 행에서 근거 열을 꺼냅니다. Jupyter는 저장된 글을 Markdown으로 표시합니다.

실행 결과

```
[9]: # MD의 표·제목 표시를 노트북 화면에서 읽기 좋게 보여 주는 도구입니다.
     from IPython.display import Markdown

     # 첫 행의 근거 본문을 표시합니다. iloc[0]은 첫 번째 행을 뜻합니다.
     Markdown(hits.iloc[0]["근거"])
```

[9]: SK하이닉스 2025Q3 III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서

연결 포괄손익계산서

제 78 기 3분기 2025.01.01 부터 2025.09.30 까지

제 77 기 3분기 2024.01.01 부터 2024.09.30 까지

(단위 : 백만원)

매출액 (주4,21,28)

검색 노트북 코드 9 · 실행 결과

확인할 점

iloc[0]
결과표의 첫 번째 행입니다.
첫 후보
2025Q3_0196
확인할 항목
절 제목, 표 제목, 기간, 단위입니다.
다음 확인
매출액 숫자는 청크 본문과 원문 파일에서 읽습니다.

주의 이 Markdown 화면에는 매출액 숫자 열이 나타나지 않습니다. 다음 화면의 원문으로 이어 확인합니다.

다음 → 같은 ID의 청크 본문을 확인합니다

같은 ID의 청크 본문을 확인합니다

검색 1위 2025Q3_0196을 chunks.jsonl에서 읽은 진단 화면입니다. 앞 교안에서 저장한 청크와 연결해 봅니다.

원문 확인

```
id      : 2025Q3_0196
source  : 000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md
period  : 2025Q3
section : III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서
text 글자 수: 910
SK하이닉스 2025Q3
III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서
| 연결 포괄손익계산서 |
| --- |
| 제 78 기 3분기 2025.01.01 부터 2025.09.30 까지 |
| 제 77 기 3분기 2024.01.01 부터 2024.09.30 까지 |
| (단위 : 백만원) |
| | 제 78 기 3분기 3개월 | 제 78 기 3분기 누적 | 제 77 기 3분기 3개월 | 제 77 기 3분기 누적 |
| --- | --- | --- | --- | --- |
```

진단 화면 5 · 청크 본문 · 실행 결과

확인할 점

청크 파일의 필드
id, source, period, section, text

화면에 보이는 부분
본문 윗부분의 절·표 제목, 기간, 단위, 열 제목입니다.

매출액 행
21~22쪽의 원문 화면에서 확인합니다.

구분

청크 파일의 다섯 필드와 DB의 metadata 세 필드는 다릅니다. 이 화면은 학습 코드 외의 보충 진단입니다.

다음 → 출처 파일에서 기간과 표의 기준을 확인합니다

출처 파일에서 기간과 표의 기준을 확인합니다

청크의 source가 가리키는 실습용 MD 파일입니다. 왼쪽 숫자는 원문 파일의 줄 번호입니다.

원문 확인

```
538 ### 2. 연결재무제표
539
540 ### 2-2. 연결 포괄손익계산서
541
542 ¦ 연결 포괄손익계산서 ¦
543 ¦ --- ¦
544 ¦ 제 78 기 3분기 2025.01.01 부터 2025.09.30 까지 ¦
545 ¦ 제 77 기 3분기 2024.01.01 부터 2024.09.30 까지 ¦
546 ¦ (단위 : 백만원) ¦
547
548 ¦  ¦ 제 78 기 3분기<br>3개월 ¦ 제 78 기 3분기<br>누적 ¦ 제 77 기 3분기<br>3개월 ¦ 제 77 기
    3분기<br>누적 ¦
549 ¦ --- ¦ --- ¦ --- ¦ --- ¦ --- ¦
550 ¦ 매출액 (주4,21,28) ¦ 24,448,929 ¦ 64,320,022 ¦ 17,573,069 ¦ 46,425,925 ¦
551 ¦ 매출원가 (주23,28) ¦ 10,419,829 ¦ 28,205,607 ¦ 8,401,685 ¦ 24,963,432 ¦
552 ¦ 매출총이익 ¦ 14,029,100 ¦ 36,114,415 ¦ 9,171,384 ¦ 21,462,493 ¦
```

원문 MD 538~552행 · 기존 캡처

확인할 점

540행 절 제목
542행 표 제목
544~545행 두 기간
546행 단위: 백만원
548행 네 열의 제목
550행 매출액

연결은 회사와 연결 대상 회사를 묶어 집계한 기준입니다.

확인

3개월은 그 분기 석 달, 누적은 1월부터 분기 말까지입니다. 백만원 단위의 1은 1,000,000원입니다.

다음 → 같은 매출액 행에서 질문에 맞는 열을 읽습니다

같은 매출액 행에서 질문에 맞는 열을 읽습니다

이번 질문은 2025년 3분기 3개월간의 연결 매출액입니다. 숫자와 함께 기간·열 제목·단위를 확인합니다.

원문

```
546 ¦ (단위 : 백만원) ¦
547
548 ¦  ¦ 제 78 기 3분기<br>3개월 ¦ 제 78 기 3분기<br>누적 ¦ 제 77 기 3분기<br>3개월 ¦ 제 77 기 3분기<br>누적 ¦
549 ¦ --- ¦ --- ¦ --- ¦ --- ¦ --- ¦
550 ¦ 매출액 (주4,21,28) ¦ 24,448,929 ¦ 64,320,022 ¦ 17,573,069 ¦ 46,425,925 ¦
```

원문 MD 546~550행 · 기존 캡처

읽을 때 확인

질문의 답은 제 78 기 3분기 3개월 열의 24,448,929백만원입니다.

548행 열 제목	제 78 기 3분기 3개월	제 78 기 3분기 누적	제 77 기 3분기 3개월	제 77 기 3분기 누적
550행 매출액 (주4,21,28)	24,448,929	64,320,022	17,573,069	46,425,925
뜻	2025년 7~9월(질문의 답)	2025년 1~9월 합계	2024년 7~9월	2024년 1~9월 합계

구분 64,320,022백만원은 1~9월 누적 값입니다. 모든 금액은 제공된 실습용 MD를 기준으로 읽습니다.

다음 → 1위 밖에 있는 근거도 확인합니다

1위 밖에 있는 근거도 확인합니다

Q09는 기존 실행에서 필요한 근거를 5위에서 찾았습니다. 한 후보만 읽으면 필요한 근거를 놓칠 수 있습니다.

실행 결과

[11]:

	id	question	answer	source	first_evidence_rank	passed
4	Q05	2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?	24,448,929백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
5	Q06	2025년 1월부터 9월까지 누적 연결 매출액은 얼마인가요?	64,320,022백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	2	True
6	Q07	2025년 3분기 보고서의 누적 연구개발비용 합계는 얼마인가요?	4,647,270백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
7	Q08	2025년 3분기 보고서에서 HBM4의 데이터 전송 통로 수와 동작 속도는 어떻게 ...	2,048개, 11Gbps 이상	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
8	Q09	2026년 1분기 연결 매출액은 얼마인가요?	52,576,287백만원	000660_SK하이닉스_2026Q1_원문발췌_실습용_v1.1.md	5	True

검사 노트북 코드 11 · 실행 결과

읽을 때 확인

Q09는 2026년 1분기 연결 매출액 질문입니다. 기존 두 실행에서 최초 근거 순위가 모두 5였습니다.

문항	질문	정답(answer)	최초 근거 순위	통과
Q05	2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?	24,448,929백만원	1	True
Q06	2025년 1월부터 9월까지 누적 연결 매출액은 얼마인가요?	64,320,022백만원	2	True
Q09	2026년 1분기 연결 매출액은 얼마인가요?	52,576,287백만원	5	True

확인

후보 수 K를 늘리면 읽을 자료도 늘어납니다. 이 사례만으로 모든 질문에서 K=5가 최적이라고 판단하지 않습니다.

다음 → 검색한 원문을 읽고 검사할 기준을 준비합니다

검색한 원문을 읽고 검사할 기준을 준비합니다

검색 노트북은 저장된 원문 조각을 돌려줍니다. 문항 노트북은 사람이 준비한 기준으로 그 검색 결과를 검사합니다.

흐름

단계	담당	받거나 만드는 것
검색 노트북	프로그램·ChromaDB	질문과 기간을 지정하고, 원문 후보 5개와 출처·거리를 확인합니다.
문항 노트북	프로그램	사람이 쓴 질문·정답·필수 근거·출처·기간을 CSV로 저장합니다.
검사 노트북	프로그램	올바른 출처의 한 청크에서 필수 문구를 찾고, 순위와 지표를 저장합니다.

확인할 내용 원문의 기간·열 제목·단위를 읽고 답에 해당하는 값을 확인합니다. 기간 자동 추출이나 하이브리드 검색은 현재 학습 범위에 포함하지 않습니다.

범위 현재 검색·문항·검사 노트북에는 생성 모델 호출이 없습니다. 답변 생성·챗봇·에이전트·재순위화는 별도 후속 구성입니다.

다음 → 질문 한 문항의 번호·기간·질문·정답을 적습니다

질문 한 문항의 번호·기간·질문·정답을 적습니다

평가 문항은 원문을 확인해 미리 작성되어 있습니다. 문항 노트북은 questions 목록에 문항을 하나씩 추가합니다.

실행 결과

```
[7]: # Q05: 원문을 기준으로 준비한 문항 하나를 기존 목록의 끝에 추가합니다.
questions.append({
    # 다른 문항과 구분할 이름입니다. 답이나 청크 ID와는 다릅니다.
    "id": "Q05",
    # 05에서 이 기간의 문서만 찾는 조건으로 사용합니다.
    "period": "2025Q3",
    # 실제 검색 모델에 전달할 질문 문장입니다.
    "question": "2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?",
    # 사람이 결과를 읽고 확인할 정답입니다. 검색 입력으로 보내지 않습니다.
    "answer": "24,448,929백만원",
```

문항 노트북 코드 7 · 1~10행 · 실행 결과

확인할 점

id
문항 번호 Q05입니다. 청크 ID와 다릅니다.

period
검색에 쓸 기간 2025Q3입니다.

question
검색할 질문입니다.

answer
사람이 확인할 정답 금액입니다.

준비 문항 노트북 코드 2가 빈 목록을 만들고 코드 3~22가 20문항을 추가합니다. answer는 검색 입력에 넣지 않습니다.

다음 → 한 청크에서 확인할 문구와 출처를 적습니다

한 청크에서 확인할 문구와 출처를 적습니다

같은 문항의 뒷부분입니다. evidence는 필수 근거 문구 목록이고, source는 정답을 확인한 원문 파일명입니다.

실행 결과

```
    # 정답 근거가 갖춰야 할 원문 문구들입니다. 05가 한 청크에서 모두 찾는지 비교합니다.
    "evidence": [
        "24,448,929",
        "매출액",
        "3개월",
        "단위 : 백만원"
    ],
    # 정답을 확인한 원문 파일입니다. 검색 후 출처를 채점할 때만 사용합니다.
    "source": "000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md"
})
```

문항 노트북 코드 7 · 11~20행 · 실행 결과

확인할 점

Q05의 필수 문구
24,448,929
매출액
3개월
단위 : 백만원
검사 조건
올바른 출처의 한 청크에 네 문구가 모두 있어야 합니다.

주의 셀을 반복하면 같은 문항을 다시 추가합니다. 처음부터 다시 할 때는 커널을 재시작하고 순서대로 실행합니다.

다음 → 근거 문구 목록을 글로 바꾸어 CSV로 저장합니다

근거 문구 목록을 글로 바꾸어 CSV로 저장합니다

프로그램은 evidence 목록을 JSON 문자열로 바꾸고, 문항 표를 data/rag_questions_answers.csv에 저장합니다.

실행 결과

```
[23]: # 목록 모양을 잃지 않고 한 칸에 담기 위해 JSON을 사용합니다.
      import json

      # 20개 문항 각각의 evidence만 바꿉니다.
      for question in questions:
          # 목록을 JSON 글로 바꿉니다. 질문·정답·출처 내용은 변경하지 않습니다.
          question["evidence"] = json.dumps(question["evidence"], ensure_ascii=False)
```

문항 노트북 코드 23 · 실행 결과

```
[25]: # 행 번호 열은 빼고, 한글을 Excel에서도 읽기 편한 문자 규칙으로 저장합니다.
      qa.to_csv(data_folder / "rag_questions_answers.csv", index=False, encoding="utf-8-sig")
```

문항 노트북 코드 25 · 실행 결과

확인할 점

코드 23
json.dumps로 목록을 JSON 글로 바꿉니다.

코드 24
저장 전 문항 표를 확인합니다. 다음 화면에서 봅니다.

코드 25
행 번호는 빼고, 한글을 읽기 쉬운 utf-8-sig로 저장합니다.

주의 변환 셀을 두 번 실행하면 이중 변환될 수 있습니다. CSV는 검색용 원문 DB와 별도의 평가 기준 파일입니다.

다음 → 20문항의 질문과 정답이 짝지어졌는지 봅니다

20문항의 질문과 정답이 짝지어졌는지 봅니다

문항 노트북 코드 24의 확인 화면입니다. 표에서 질문과 정답을 확인한 뒤 코드 25로 저장합니다.

실행 결과

```
[24]: # 문항 목록을 표와 CSV로 다루는 도구입니다.
import pandas as pd

# 문항 한 개를 한 행으로 만든 표입니다.
qa = pd.DataFrame(questions)
# 검토하기 쉬운 네 열만 화면에 표시합니다. 나머지 열을 삭제하지 않습니다.
qa[["id", "period", "question", "answer"]]
```

[24]:

	id	period	question	answer
0	Q01	2025FY	2025년 연간 SK하이닉스의 연결 매출액은 얼마인가요?	97,146,675백만원
1	Q02	2025FY	2025년 연간 SK하이닉스의 연결 영업이익은 얼마인가요?	47,206,319백만원
2	Q03	2025FY	2025년 연구개발비용 합계와 매출액 대비 연구개발비 비율은 얼마인가요?	6,732,527백만원, 6.9%
3	Q04	2025FY	2025년 연결 현금흐름표의 영업활동 현금흐름은 얼마인가요?	53,373,126백만원
4	Q05	2025Q3	2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?	24,448,929백만원
5	Q06	2025Q3	2025년 1월부터 9월까지 누적 연결 매출액은 얼마인가요?	64,320,022백만원

문항 노트북 코드 24 · 실행 결과

확인할 점

표에 보이는 열
id, period, question, answer

함께 남아 있는 열
evidence, source

행 확인
Q01~Q20이 각각 한 번씩 있어야 합니다. 화면은 Q06까지 발췌했습니다.

연결

다음 검사 노트북은 이 CSV를 읽습니다. 검사 노트북 코드 1~4에서 작업 폴더·설정·기존 컬렉션·CSV를 먼저 준비합니다.

다음 → CSV의 근거 문구를 비교하기 쉽게 정리합니다

CSV의 근거 문구를 비교하기 쉽게 정리합니다

프로그램은 json.loads로 JSON 글을 다시 목록으로 읽고, 표시 방식과 공백 차이를 정리합니다.

실행 결과

```
[5]: # 각 문항의 필수 근거 문구를 비교하기 쉬운 모양으로 준비합니다.
for question in questions:
    # CSV 한 칸의 JSON 글을 다시 문구 목록으로 읽습니다.
    pieces = json.loads(question["evidence"])
    # 정리한 문구들을 담을 빈 목록입니다.
    cleaned = []
    # 필수 근거 문구를 하나씩 처리합니다.
    for piece in pieces:
        # MD 표시용 역슬래시를 빼고 <br>를 공백으로 바꿉니다.
        piece = piece.replace("\\", "").replace("<br>", " ")
        # 공백으로 나눈 뒤 빈 글로 이어 붙여 모든 공백을 없앱니다.
        cleaned.append("".join(piece.split()))
    # 이후 채점에서는 정리된 근거 목록을 사용합니다.
    question["evidence"] = cleaned
```

검사 노트북 코드 5 · 실행 결과

확인할 점

같은 정리 규칙

MD 표시용 역슬래시를 뺍니다.

를 공백으로 바꿉니다.
모든 공백을 없앱니다.

목적

공백 표기 차이 때문에 같은 근거를 놓치지 않게 합니다.

구분

비교용 글만 정리합니다. 원문 파일을 수정하지 않습니다. json.dumps와 json.loads의 방향을 구분합니다.

다음 → 필수 문구가 지정한 원문에 있는지 검사합니다

필수 문구가 지정한 원문에 있는지 검사합니다

프로그램은 각 문항의 source 파일을 읽고 같은 정리 규칙을 적용합니다. 검색 전에 평가 기준의 문구를 확인합니다.

실행 결과

```
[6]: # 문항마다 정답 출처 파일을 확인합니다. 검색어로 넘기는 단계가 아닙니다.
for question in questions:
    # data 폴더 위치와 원문 파일명을 /로 연결합니다.
    path = data_folder / question["source"]
    # 지정된 원문 MD를 읽습니다.
    original = path.read_text(encoding="utf-8-sig")
    # 필수 문구에 사용한 것과 같은 표시 정리 규칙을 적용합니다.
    original = original.replace("\\", "").replace("<br>", " ")
    # 줄바꿈·띄어쓰기를 제거한 비교용 글로 만듭니다.
    original = "".join(original.split())
    # 필수 문구를 하나라도 빠뜨리지 않고 확인합니다.
    for piece in question["evidence"]:
        # in은 포함 여부를 검사합니다. 문구가 없으면 문항 ID와 함께 멈춥니다.
        assert piece in original, question["id"]
```

검사 노트북 코드 6 · 실행 결과

확인할 점

assert piece in original
문구가 원문에 있는지 검사합니다.

실패하면

문항 ID와 함께 멈춥니다.

확인할 것

문구, 원문 파일, CSV 버전을 대조합니다.

구분

원문에 문구가 있다는 확인입니다. 검색 성공이나 표 의미를 정확히 해석했다는 확인은 아닙니다.

다음 → 20개 질문을 같은 모델로 임베딩합니다

20개 질문을 같은 모델로 임베딩합니다

프로그램은 검색 지시문과 question을 합친 글을 모아 Ollama에 보냅니다. 모델은 입력 순서대로 숫자 목록을 돌려줍니다.

실행 결과

```
[7]: # 임베딩할 질문 글들만 담을 목록입니다.
query_texts = []
# CSV 문항 순서를 그대로 유지합니다.
for question in questions:
    # 공통 검색 지시문과 질문만 합칩니다. answer·evidence·source는 사용하지 않습니다.
    query_texts.append(settings["query_prefix"] + question["question"])
```

검사 노트북 코드 7 · 실행 결과

```
[8]: # 질문도 저장 때와 같은 Ollama로 임베딩합니다.
import ollama

# 모델 이름은 02에서 저장한 설정을 그대로 사용합니다.
response = ollama.embed(model=settings["model"], input=query_texts, truncate=False)
# 질문 순서와 같은 순서로 반환된 숫자 목록들을 받습니다.
query_vectors = response.embeddings
```

검사 노트북 코드 8 · 실행 결과

확인할 점

입력
검색 지시문 + 질문 20개
모델
저장 노트북의 settings["model"]
결과
질문 순서에 대응하는 숫자 목록 20개
비교 기준
answer, evidence, source는 검색 후 검사에 씁니다.

연결

검색 노트북은 질문 한 개, 검사 노트북은 20개를 임베딩합니다. 정답과 정답 출처를 검색어에 넣지 않습니다.

다음 → 문항마다 해당 보고기간에서 후보를 찾습니다

문항마다 해당 보고기간에서 후보를 찾습니다

프로그램은 같은 번호의 질문과 숫자 목록을 함께 사용합니다. ChromaDB는 각 문항의 period 조건에서 최대 5개를 찾습니다.

실행 결과

[9]:
```
# 20개 문항의 번호 0~19를 차례대로 사용합니다.
for number in range(len(questions)):
    # 이번 순서에 해당하는 문항을 꺼냅니다.
    question = questions[number]
    # 이 문항에 대한 검색 응답을 받습니다.
    found = collection.query(
        # 같은 번호의 질문 임베딩을 사용합니다.
        query_embeddings=[query_vectors[number]],
        # 이 문항의 보고기간 범위에서 최대 다섯 개를 찾습니다.
        where={"period": question["period"]}, n_results=5,
        # 채점과 확인에 필요한 원문·출처·거리도 함께 받습니다.
        include=["documents", "metadatas", "distances"]
    )
    # 나중에 이 문항의 정답과 비교할 수 있도록 응답을 보관합니다.
    question["search"] = found
```

검사 노트북 코드 9 · 실행 결과

확인할 점

number
0~19로 같은 문항을 가리킵니다.

where
question["period"]로 범위를 정합니다.

question["search"]
그 문항의 검색 응답 전체를 보관합니다.

확인

질문 20개를 같은 기간으로 검색하지 않습니다. 문항마다 CSV에 적힌 보고기간을 사용합니다.

다음 → 근거 순위를 0으로 두고 후보 검사를 시작합니다

근거 순위를 0으로 두고 후보 검사를 시작합니다

프로그램은 문항마다 first_evidence_rank를 0으로 놓고, 그 문항의 첫 후보부터 차례대로 확인합니다.

실행 결과

```
[10]: # 각 질문에 붙여 둔 실제 검색 결과를 채점합니다.
for question in questions:
    # 이 문항에서 받은 다섯 결과를 꺼냅니다.
    found = question["search"]
    # 아직 찾지 못했다는 표시로 0을 넣습니다.
    question["first_evidence_rank"] = 0
    # 첫 결과부터 순서대로 확인합니다.
    for number in range(len(found["ids"][0])):
```

검사 노트북 코드 10 · 1~8행 · 실행 결과

확인할 점

0의 뜻
아직 기준에 맞는 근거를 찾지 못했다는 표시입니다.

found
검사 노트북 코드 9의 검색 응답입니다.

number
후보 다섯 개의 번호 0~4입니다.

구분 first_evidence_rank의 0은 검색 순위 0위라는 뜻이 아닙니다. 검사 전의 시작값입니다.

다음 → 후보의 글을 정리하고 원문 출처를 비교합니다

후보의 글을 정리하고 원문 출처를 비교합니다

프로그램은 후보 하나의 text를 꺼내 정리하고, 후보의 source가 문항의 정답 출처와 같은지 비교합니다.

실행 결과

```
        # 현재 순위의 원문 글을 꺼냅니다.
        text = found["documents"][0][number]
        # 정답 근거에 적용한 것과 같은 표시 정리를 합니다.
        text = text.replace("\\", "").replace("<br>", " ")
        # 공백 차이 때문에 근거를 놓치지 않도록 비교용 글을 만듭니다.
        text = "".join(text.split())
        # 현재 검색 결과의 실제 원문 파일명을 꺼냅니다.
        source = found["metadatas"][0][number]["source"]
        # 정답 출처와 같은지 먼저 비교합니다. ==는 대입이 아니라 같음 비교입니다.
        matched = source == question["source"]
```

검사 노트북 코드 10 · 9~18행 · 실행 결과

확인할 점

같은 정리 규칙
근거 문구와 후보 글에 똑같이 적용합니다.

source 비교
==는 두 파일명이 같은지 비교합니다.

matched
출처가 맞으면 True로 시작합니다.

확인

출처가 같아도 검사는 끝나지 않았습니다. 다음 부분에서 필수 문구가 모두 있는지 확인합니다.

다음 → 모든 필수 문구를 찾으면 최초 순위를 기록합니다

모든 필수 문구를 찾으면 최초 순위를 기록합니다

올바른 출처의 한 청크에 evidence의 모든 문구가 있으면, 프로그램은 그 후보의 순위를 기록합니다.

실행 결과

```
        # 현재 한 청크 안에서 모든 필수 문구를 확인합니다.
        for piece in question["evidence"]:
            # 필수 문구가 하나라도 없으면 완전한 근거가 아닙니다.
            if piece not in text:
                # 이 청크는 통과하지 못했다고 표시합니다.
                matched = False
        # 출처와 모든 문구가 맞는 경우에만 아래 두 줄을 실행합니다.
        if matched:
            # 0부터 센 번호를 사람이 읽는 순위 1, 2, ...로 바꿉니다.
            question["first_evidence_rank"] = number + 1
            # 이 문항에서 첫 근거를 찾았으므로 다음 결과는 더 검사하지 않습니다.
            break
```

검사 노트북 코드 10 · 19~30행 · 실행 결과

확인할 점

문구 하나가 없으면
matched는 False입니다.

모두 있으면
number + 1을 순위로 기록합니다.

break
이 문항의 남은 후보 검사를 끝냅니다.

찾지 못하면
순위 0이 남습니다.

주의

서로 다른 청크의 문구를 합쳐 통과시키지 않습니다. break는 20문항 전체 검사를 끝내는 코드가 아닙니다.

다음 → 문항별 최초 근거 순위와 통과 여부를 표로 만듭니다

문항별 최초 근거 순위와 통과 여부를 표로 만듭니다

프로그램은 questions를 results 표로 바꾸고, first_evidence_rank가 0보다 큰지 비교해 passed를 만듭니다.

실행 결과

[11]:
```
# 문항별 기록을 표로 바꿉니다.
results = pd.DataFrame(questions)
# 최종 확인에 필요한 열만 남긴 결과표를 만듭니다.
results = results[["id", "question", "answer", "source", "first_evidence_rank"]]
# 각 행의 순위가 0보다 크면 True, 아니면 False를 기록합니다.
results["passed"] = results["first_evidence_rank"] > 0
# 완성된 20행 결과표를 화면에 보여 줍니다.
results
```

검사 노트북 코드 11 · 실행 결과

확인할 점

answer
사람이 미리 쓴 정답입니다.

first_evidence_rank
기준을 처음 만족한 후보의 순위입니다.

passed
근거 순위가 1 이상이면 True입니다.

구분 표는 상위 5개에서 필수 문구를 찾았는지 보여 줍니다. 생성 답변의 정답 여부나 원문의 최신성은 평가하지 않습니다.

다음 → 문항별로 근거를 처음 찾은 순위를 읽습니다

문항별로 근거를 처음 찾은 순위를 읽습니다

2026-09-21 실행에서는 20문항 모두 passed=True였습니다. Q05·Q06·Q09는 각각 1위·2위·5위에서 근거를 찾았습니다.

실행 결과

[11]:

	id	question	answer	source	first_evidence_rank	passed
4	Q05	2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?	24,448,929백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
5	Q06	2025년 1월부터 9월까지 누적 연결 매출액은 얼마인가요?	64,320,022백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	2	True
6	Q07	2025년 3분기 보고서의 누적 연구개발비용 합계는 얼마인가요?	4,647,270백만원	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
7	Q08	2025년 3분기 보고서에서 HBM4의 데이터 전송 통로 수와 동작 속도는 어떻게 ...	2,048개, 11Gbps 이상	000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md	1	True
8	Q09	2026년 1분기 연결 매출액은 얼마인가요?	52,576,287백만원	000660_SK하이닉스_2026Q1_원문발췌_실습용_v1.1.md	5	True

검사 노트북 코드 11 · 실행 결과

읽을 때 확인

Q05는 3분기 3개월, Q06은 1~9월 누적, Q09는 2026년 1분기의 연결 매출액 질문입니다.

문항	질문	정답(answer)	최초 근거 순위	통과
Q05	2025년 3분기 3개월간 SK하이닉스의 연결 매출액은 얼마인가요?	24,448,929백만원	1	True
Q06	2025년 1월부터 9월까지 누적 연결 매출액은 얼마인가요?	64,320,022백만원	2	True
Q09	2026년 1분기 연결 매출액은 얼마인가요?	52,576,287백만원	5	True

확인 passed=False는 검사한 상위 5개에 기준을 만족하는 청크가 없다는 뜻입니다. 전체 원문에 답이 없다는 뜻은 아닙니다.

다음 → 전체 문항 중 근거를 찾은 비율을 계산합니다

전체 문항 중 근거를 찾은 비율을 계산합니다

Hit@1과 Hit@5는 같은 20문항을 분모로 씁니다. 분자는 어디까지 읽었을 때 근거를 찾았는지에 따라 달라집니다.

실행 결과

```
[12]: # 최종 결과를 이름과 값으로 모읍니다.
summary = {
    # 전체 문항 수와 passed=True인 문항 수입니다.
    "questions": len(results), "passed": int(results["passed"].sum()),
    # 순위가 정확히 1인 문항의 비율을 계산합니다.
    "hit_at_1": float((results["first_evidence_rank"] == 1).mean()),
    # 상위 다섯 개에서 근거를 찾은 문항의 비율입니다.
    "hit_at_5": float(results["passed"].mean()),
```

검사 노트북 코드 12 · 1~8행 · 실행 결과

mean()은 True를 1, False를 0으로 보고 평균을 냅니다. questions는 전체 수, passed는 통과 문항 수입니다.

Hit@1 첫 후보에서 근거를 찾은 문항 ÷ 전체
16 ÷ 20 = 0.8 (80%)

Hit@5 상위 5개에서 근거를 찾은 문항 ÷ 전체
20 ÷ 20 = 1.0 (100%)

범위 여기서 5는 현재 코드의 후보 개수입니다. 지표는 지정한 기간 안에서 필수 근거 문구를 발견한 비율입니다.

다음 → summary에서 문항 수와 비율을 확인합니다

summary에서 문항 수와 비율을 확인합니다

프로그램이 만든 summary에는 전체 문항 수와 통과 수, Hit@1, Hit@5, 사용한 모델과 검사 범위가 들어 있습니다.

실행 결과

```
    # 어떤 모델과 보관함으로 검사했는지 기록합니다.
    "model": settings["model"], "collection": settings["collection"],
    # 점수의 의미를 나중에 잘못 해석하지 않도록 평가 범위를 기록합니다.
    "scope": "기간을 지정한 검색의 근거 발견률. 생성 답변 평가 아님."
}
# 계산한 요약 값을 화면에 표시합니다.
summary
```

```
[12]: {'questions': 20,
 'passed': 20,
 'hit_at_1': 0.8,
 'hit_at_5': 1.0,
 'model': 'qwen3-embedding:0.6b',
 'collection': 'skhynix_basic',
 'scope': '기간을 지정한 검색의 근거 발견률. 생성 답변 평가 아님.'}
```

검사 노트북 코드 12 · 9~15행 · 실행 결과

확인할 점

questions 20
passed 20
hit_at_1 0.8
hit_at_5 1.0
해석
첫 후보에서 16문항, 상위 5개에서 20문항의 근거를 찾았습니다.

구분

현재 20문항의 근거 발견률입니다. RAG 전체 정확도 100% 또는 생성 답변이 모두 정답이라는 뜻은 아닙니다.

다음 → 문항별 결과와 검색 응답을 파일로 저장합니다

문항별 결과와 검색 응답을 파일로 저장합니다

프로그램은 노트북을 닫은 뒤에도 결과를 확인할 수 있도록 results와 검색 응답을 artifacts 폴더에 남깁니다.

실행 결과

```
[13]: # 실제 검사 결과표를 저장합니다. index=False로 화면용 행 번호 열은 제외합니다.
      results.to_csv(output_folder / "test_results.csv", index=False, encoding="utf-8-sig")
```

검사 노트북 코드 13 · 실행 결과

```
[14]: # 문항별 원문 검색 응답을 모을 목록입니다.
      searches = []
      # CSV와 같은 문항 순서로 기록합니다.
      for question in questions:
          # 이 문항의 실제 검색 응답 전체를 추가합니다.
          searches.append(question["search"])
      # 검색 원문 기록 파일을 쓰기 모드로 엽니다.
      with (output_folder / "retrieval_results.json").open("w", encoding="utf-8") as file:
          # 한글과 들여쓰기를 유지해 실제 검색 내용을 저장합니다.
          json.dump(searches, file, ensure_ascii=False, indent=2)
```

검사 노트북 코드 14 · 실행 결과

확인할 점

test_results.csv
검사 노트북 코드 11의 문항별 결과표입니다.

retrieval_results.json
문항 순서대로 저장한 검색 응답입니다.

연결 기준
응답에 문항 ID가 자동으로 들어가지 않아 CSV의 행 순서로 연결합니다.

확인

검색 응답에는 후보 ID·원문·이름표·거리가 있습니다. CSV와 JSON을 같은 실행의 파일끼리 읽습니다.

다음 → 전체 지표를 저장하고 마지막 조건을 검사합니다

전체 지표를 저장하고 마지막 조건을 검사합니다

프로그램은 summary를 test_summary.json으로 저장하고, assert로 컬렉션 수와 문항 수, 통과 여부를 확인합니다.

실행 결과

[15]:
```
# 전체 점수 요약을 남길 파일을 엽니다.
with (output_folder / "test_summary.json").open("w", encoding="utf-8") as file:
    # 앞에서 계산한 summary를 그대로 기록합니다.
    json.dump(summary, file, ensure_ascii=False, indent=2)
```

검사 노트북 코드 15 · 실행 결과

[16]:
```
# 저장된 개수가 02가 기록한 개수와 같은지 확인합니다.
assert collection.count() == settings["chunk_count"]
# 요청한 최소 문항 수 10개를 충족하는지 확인합니다.
assert len(results) >= 10
# 한 문항이라도 근거를 못 찾았으면 멈춥니다. 성공하면 별도 출력이 없습니다.
assert results["passed"].all()
```

검사 노트북 코드 16 · 실행 결과

확인할 점

마지막 검사
DB 항목 수 = 설정의 800
문항 수는 10개 이상
모든 문항이 passed=True
별도 확인
정확히 20개인지, ID가 유일한지, 파일 내용이 맞는지도 봅니다.

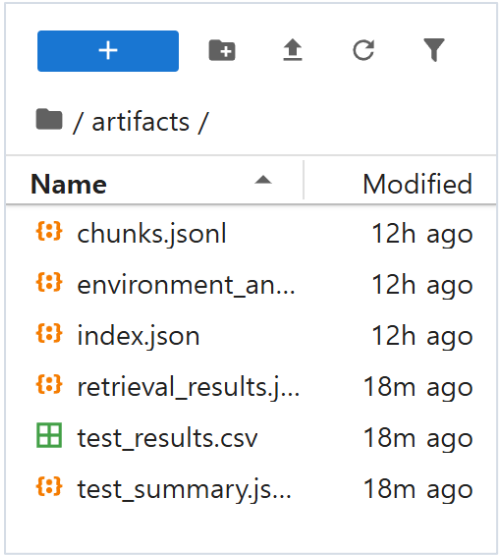
확인 assert는 성공하면 별도 문구를 출력하지 않습니다. 셀 실행 완료, 오류 유무, 저장 파일 내용을 함께 확인합니다.

다음 → 저장한 결과 파일 세 개를 확인합니다

저장한 결과 파일 세 개를 확인합니다

artifacts 폴더에서 수정 시각과 파일 내용을 함께 확인합니다. 파일 이름만으로 최신 실행 여부를 판단하지 않습니다.

저장 결과



artifacts 폴더 · 기존 캡처

파일	내용	만드는 코드
test_results.csv	문항별 결과표: id, question, answer, source, first_evidence_rank, passed	코드 13
retrieval_results.json	문항 순서대로 저장한 검색 원문 응답(ID·원문·이름표·거리). 문항 ID는 CSV의 행 순서로 연결	코드 14
test_summary.json	전체 지표: questions, passed, hit_at_1, hit_at_5, model, collection, scope	코드 15
index.json · chunks.jsonl	저장·청크 노트북이 만든 설정과 청크 파일입니다. 검사 노트북은 읽기만 하고 바꾸지 않습니다.	저장 노트북 코드 11 · 청크 노트북

test_summary.json을 열어 questions 20, passed 20, hit_at_1 0.8, hit_at_5 1.0을 확인합니다.

연결 index.json은 저장 노트북의 설정, chunks.jsonl은 청크 노트북의 청크입니다. 검사 노트북은 이 앞 단계 파일을 바꾸지 않습니다.

다음 → 활동. 기간·열·순위를 구분해 봅니다

활동. 기간·열·순위를 구분해 봅니다

아래 자료를 읽고 질문에 답합니다. 가정 활동은 원리를 확인하기 위한 상황이며 실제 오류 화면이 아닙니다.

활동 1 · 가정 활동

질문은 2025Q3인데 period가 2026H1인 가정에서 무엇을 먼저 고쳐야 합니까?

자료: 보고기간 이름표는 2025FY·2025Q3·2026Q1·2026H1·20260722 다섯 가지이고, where={"period": period}는 그 기간의 청크만 검색합니다.

활동 2 · 실제 자료

Q3 3개월 매출액으로 64,320,022백만원을 선택하면 왜 맞지 않습니까?

자료: 550행 | 매출액 (주4,21,28) | 24,448,929 | 64,320,022 | 17,573,069 | 46,425,925 | 548행 열 제목: 제 78 기 3분기 3개월 | 제 78 기 3분기 누적 | 제 77 기 3분기 3개월 | 제 77 기 3분기 누적

활동 3 · 실제 결과

Q09에서 1위만 읽으면 어떤 근거를 놓칠 수 있습니까?

자료: Q09 '2026년 1분기 연결 매출액은 얼마인가요?', 정답 52,576,287백만원, 최초 근거 순위 5 (기존 저장 기록과 신규 실행 모두)

진행

활동 1~3의 답과 이유는 45쪽에 있습니다. 먼저 어떤 자료를 확인했는지 함께 말합니다.

다음 → 활동. 통과 조건과 지표의 뜻을 설명합니다

활동. 통과 조건과 지표의 뜻을 설명합니다

프로그램이 무엇을 검사했는지에 맞춰 답합니다. 가정 활동과 기존 실행 결과를 구분해서 읽습니다.

활동 4 · 가정 활동

한 후보에는 금액이 있고 다른 후보에는 단위가 있는 가정에서 현재 검사는 통과합니까?

자료: 검사 노트북 코드 10은 후보 하나의 text 안에서 evidence의 모든 문구를 확인합니다. Q05 evidence: 24,448,929 / 매출액 / 3개월 / 단위 : 백만원

활동 5 · 실제 결과

Hit@5가 100%이면 생성 모델의 답변도 전부 정확하다고 말할 수 있습니까?

자료: summary는 passed 20/20, hit_at_5 1.0이고 scope는 '기간을 지정한 검색의 근거 발견률. 생성 답변 평가 아님.' 입니다.

활동 6 · 가정 활동

first_evidence_rank=0이면 원문 어디에도 근거가 없다고 단정할 수 있습니까?

자료: 검사 노트북 코드 10은 초기값 0으로 시작하고, 검사한 상위 5개에 통과 후보가 없으면 0이 그대로 남습니다.

진행

활동 4~6의 답과 이유는 46쪽에 있습니다. 숫자와 함께 그 숫자가 뜻하는 범위를 말합니다.

다음 → 해설. 질문의 기간과 열, 근거 순위를 확인합니다

해설. 질문의 기간과 열, 근거 순위를 확인합니다

활동 1~3의 답과 이유입니다. 앞 교안의 period 이름표가 검색 조건으로 이어지는 점도 함께 확인합니다.

활동 1 · 가정 활동

질문은 2025Q3인데 period가 2026H1인 가정에서 무엇을 먼저 고쳐야 합니까?

질문의 기간과 period를 일치시킵니다(2025Q3). 잘못된 기간 조건은 원하는 근거를 검색 범위에서 제외할 수 있습니다. 고친 뒤에는 임베딩·검색 이후 셀을 다시 실행합니다.

활동 2 · 실제 자료

Q3 3개월 매출액으로 64,320,022백만원을 선택하면 왜 맞지 않습니까?

64,320,022백만원은 같은 행의 '누적' 열 값, 즉 2025년 1~9월 합계입니다. 질문의 '3개월간'에 맞는 값은 '제 78 기 3분기 3개월' 열의 24,448,929백만원입니다.

활동 3 · 실제 결과

Q09에서 1위만 읽으면 어떤 근거를 놓칠 수 있습니까?

Q09의 필요한 근거는 5위이므로 1위만 읽으면 그 근거를 놓칠 수 있습니다. 상위 5개를 함께 읽어야 이 문항의 근거를 찾을 수 있습니다.

확인

후보 순위만으로 답을 정하지 않습니다. 질문의 기간과 원문 표의 열 제목을 함께 읽습니다.

다음 → 해설. 한 청크의 근거와 지표 범위를 구분합니다

해설. 한 청크의 근거와 지표 범위를 구분합니다

활동 4~6의 답과 이유입니다. 검사한 후보의 범위와 생성 답변 평가 여부를 구분합니다.

활동 4 · 가정 활동

한 후보에는 금액이 있고 다른 후보에는 단위가 있는 가정에서 현재 검사는 통과합니까?

통과하지 않습니다. 현재 검사는 올바른 출처의 한 청크 안에 모든 필수 문구가 있어야 통과합니다. 여러 청크의 문구를 합치지 않습니다.

활동 5 · 실제 결과

Hit@5가 100%이면 생성 모델의 답변도 전부 정확하다고 말할 수 있습니까?

말할 수 없습니다. 이번 지표는 20문항의 검색 근거 발견 검사이며 생성 답변 평가가 없습니다. 생성 모델 호출 자체가 현재 노트북에 없습니다.

활동 6 · 가정 활동

first_evidence_rank=0이면 원문 어디에도 근거가 없다고 단정할 수 있습니까?

단정할 수 없습니다. 0은 검사한 상위 5개에서 기준을 만족하는 근거를 찾지 못했다는 뜻입니다. 기간·질문·검색 후보·원문을 다시 확인해야 합니다.

마무리

통과 여부만 말하지 않고, 어떤 출처의 어떤 청크에서 무엇을 확인했는지 설명합니다.

다음 → 저장한 자료에서 근거를 찾고 결과를 설명합니다

저장한 자료에서 근거를 찾고 결과를 설명합니다

앞 교안의 저장 결과를 검색과 검사에 이어 썼습니다. 아래 내용을 직접 확인하고 자기 말로 설명하면 이 교안의 목표에 도달한 것입니다.

완료 기준

1. 질문과 보고기간을 각각 지정하고 같은 모델로 질문을 임베딩합니다.
2. 후보표의 순위·청크 ID·거리·출처를 읽고 원문을 엽니다.
3. 원문의 기간·열 제목·단위를 확인해 3개월과 누적을 구분합니다.
4. 한 청크의 출처와 필수 문구가 검사 기준임을 설명합니다.
5. first_evidence_rank·passed·Hit@1·Hit@5의 뜻을 설명합니다.
6. 결과 파일 세 개와 마지막 assert의 검사 범위를 확인합니다.

기존 실행값 800개 중 2025Q3 203개 · Q05 1위(0.177438) · Hit@1 16/20 · Hit@5 20/20

다음 단계

찾은 근거를 생성 모델에 전달하는 구현은 별도 후속 작업입니다. 현재 제공 자료는 청크·저장·검색·문항·검사 노트북입니다.

다음 → 연결이나 파일 오류가 나면 먼저 확인할 위치

연결이나 파일 오류가 나면 먼저 확인할 위치

앞 교안과 같은 순서로 증상, 확인할 위치, 다음 행동을 읽습니다. 저장소를 다시 만들기 전에 경로와 설정부터 봅니다.

증상	먼저 확인할 항목	다음 행동
index.json을 찾지 못함	Path.cwd()와 artifacts 폴더	실행 폴더가 rag_exam인지, 저장 노트북이 index.json을 저장했는지 확인합니다.
컬렉션을 찾지 못함	chroma_db 폴더 경로와 컬렉션 이름	저장소가 같은 폴더에 있는지와 설정의 이름을 확인합니다. 빈 DB를 성공 상태로 취급하지 않습니다.
Ollama에 연결하지 못함	기존 앱·서비스 실행 상태	설치된 Ollama 앱이 실행 중인지 확인합니다. 무조건 재설치하지 않습니다.
모델을 찾지 못함	ollama list의 정확한 태그	qwen3-embedding:0.6b가 준비되어 있는지 확인합니다. 다른 태그를 임의로 쓰지 않습니다.
패키지 import 실패	실제 커널의 Python 경로	Python (stock_agent_env01) 커널과 그 환경에 pandas·chromadb·ollama가 있는지 확인합니다.

연결 검색은 기존 chroma_db와 index.json을 함께 사용합니다. 설정 파일과 DB가 같은 실습 자료인지 확인합니다.

다음 → 검색이나 검사 결과가 다르면 먼저 확인할 위치

검색이나 검사 결과가 다르면 먼저 확인할 위치

질문과 기간, 실제 후보와 원문을 대조합니다. 오류를 숨기지 않고 실패한 문항과 조건을 확인합니다.

증상	먼저 확인할 항목	다음 행동
문항 중복·근거 변환 오류	셀 재실행 이력	커널을 재시작하고 처음부터 순서대로 한 번씩 실행합니다.
검색 결과의 기간이 질문과 다름	question과 period	두 입력을 맞춘 뒤 임베딩·검색 이후 셀을 다시 실행합니다.
원문 존재 assert 실패(코드 6)	문항 ID, 원문·CSV 버전, 문구	지정 원문과 필수 문구를 대조합니다. 검사문을 지우지 않습니다.
최초 근거 순위 0 또는 마지막 assert 실패	실제 후보·출처·기간·필수 문구	실패 문항의 후보와 원문을 확인합니다. 성공 결과만 남기지 않습니다.
거리만 기존 표와 다름	실행 시각·모델·필터·후보 ID	같은 실행 기록끼리 비교합니다. 소수값 차이만으로 정답 여부를 판정하지 않습니다.

마무리 설정 확인, 원문 대조, 검색 결과 해석까지가 이번 학습입니다. 필요한 화면으로 돌아가 같은 순서로 확인합니다.